Necessarily Rank-Maximal Matching

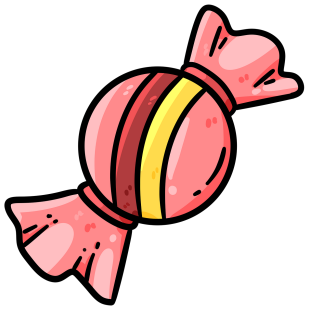
Madhav P Nair - 112301020
Devapriya Pradeep - 112301006

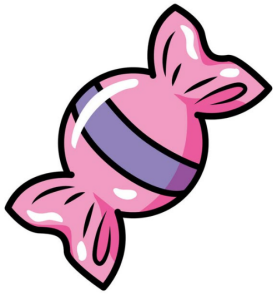
November 27, 2025

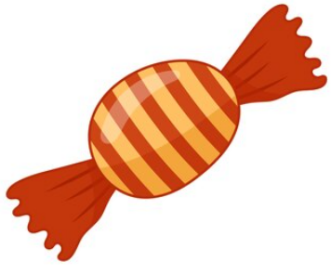
Outline

1. Introduction
2. Rank-Maximal Matching
3. Necessarily Rank-Maximal Matching
4. Results

Matching under preferences







Preference table

- agents(N) = individuals.
- objects(O) = candies.

Agents	**Preferences**			
a_1	o_1	o_2	o_4	o_3
a_2	o_1	o_2	o_3	o_4
a_3	o_3	o_4	o_2	o_1
a_4	o_3	o_1	o_4	o_2

Arbitrary Matchings

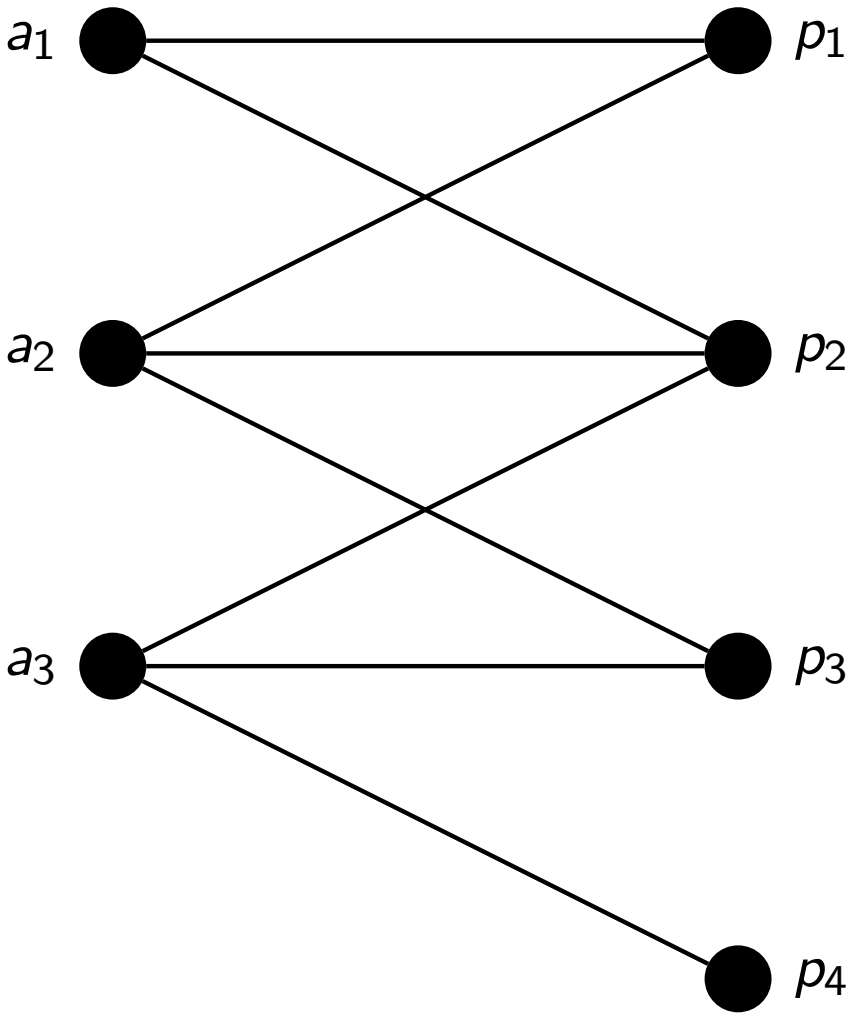
Agents	Preferences			
a_1	o_1	o_2	o_4	o_3
a_2	o_1	o_2	o_3	o_4
a_3	o_3	o_4	o_2	o_1
a_4	o_3	o_1	o_4	o_2

Agents	Preferences			
a_1	o_1	o_2	o_4	o_3
a_2	o_1	o_2	o_3	o_4
a_3	o_3	o_4	o_2	o_1
a_4	o_3	o_1	o_4	o_2

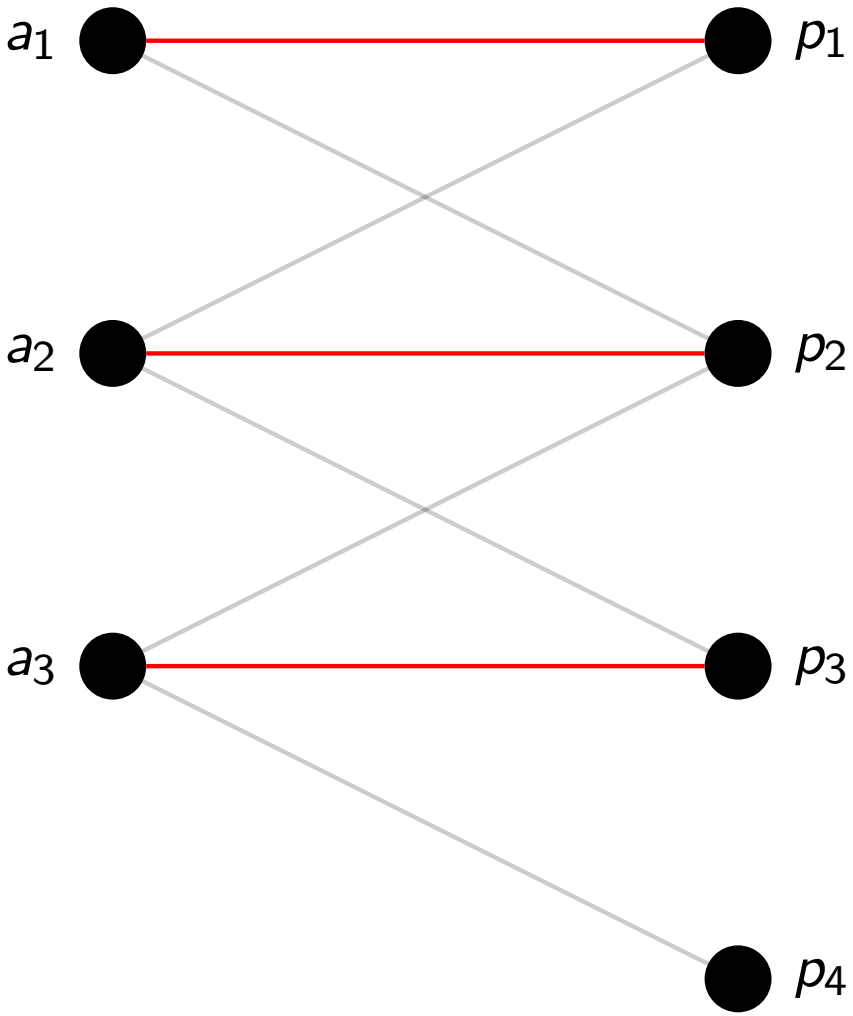
Which of these two matchings is better?

Rank-Maximal Matching

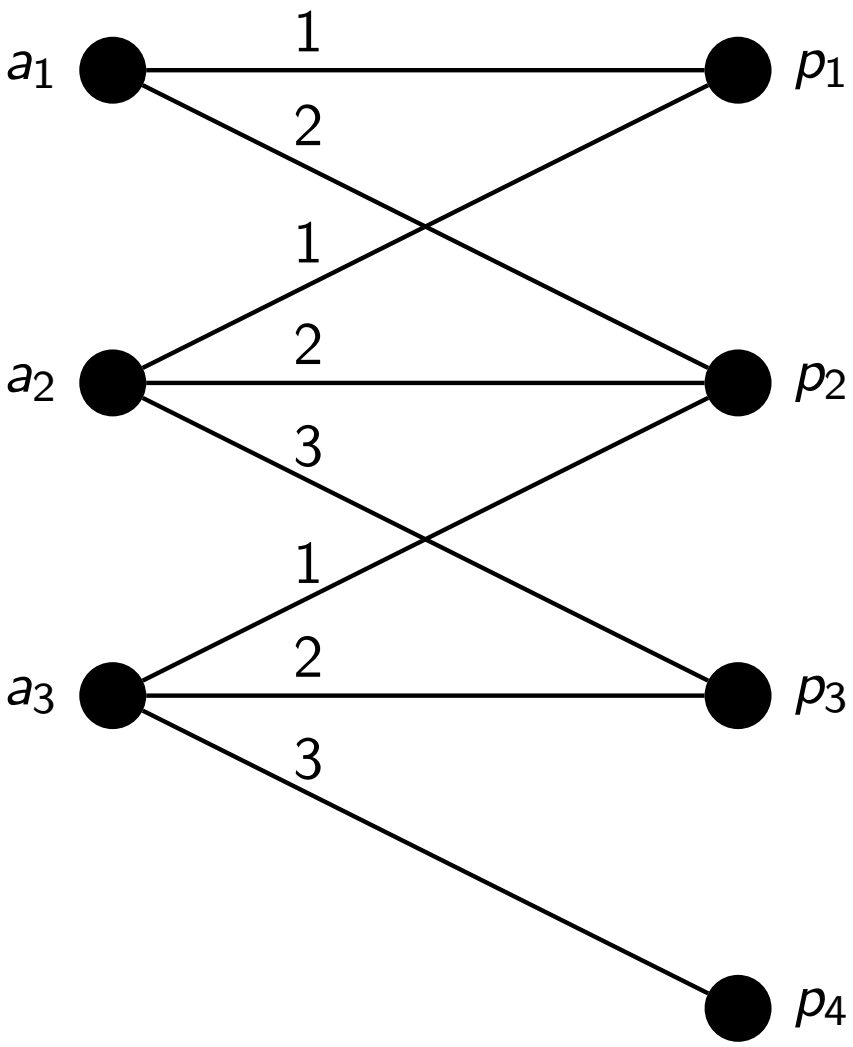
Rank-Maximal Matching



Rank-Maximal Matching



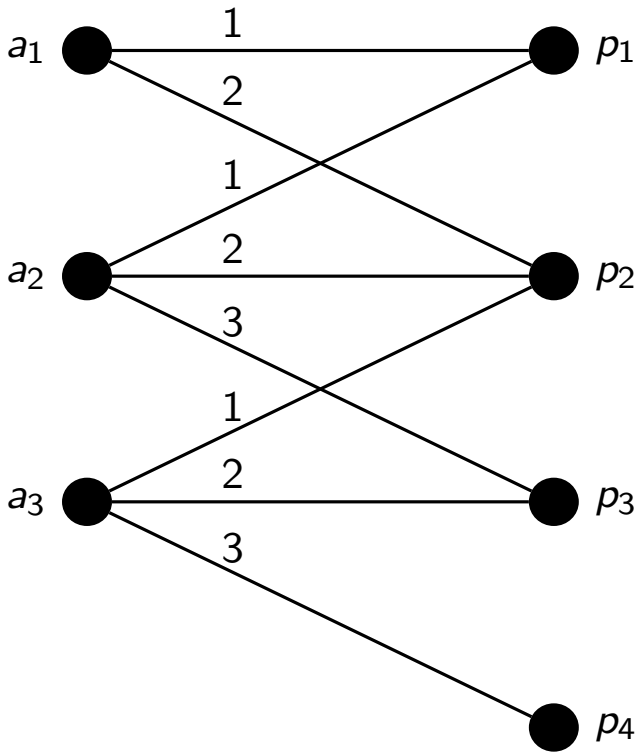
Rank-Maximal Matching



Rank-Maximal Matching

Input:

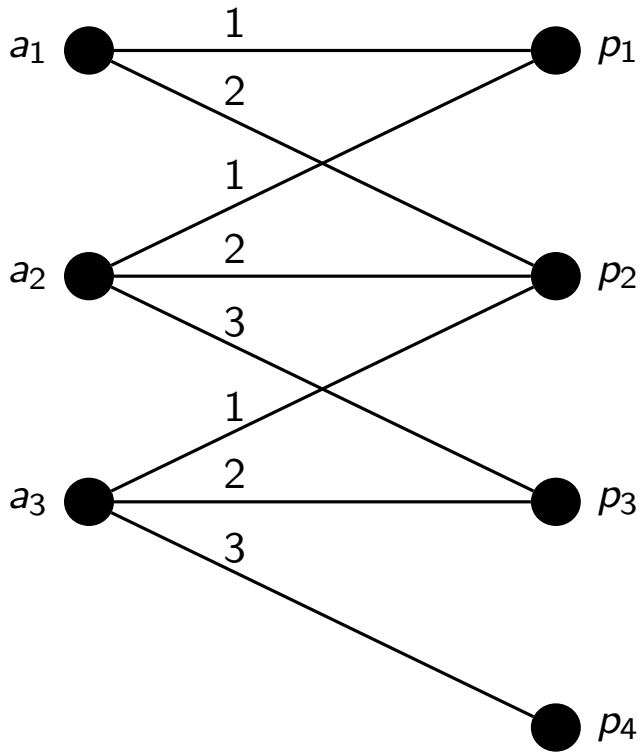
- A bipartite graph $G = (A \cup P, \mathcal{E})$
  - A denotes the set of applicants.
  - P denotes the set of posts.



Rank-Maximal Matching

Input:

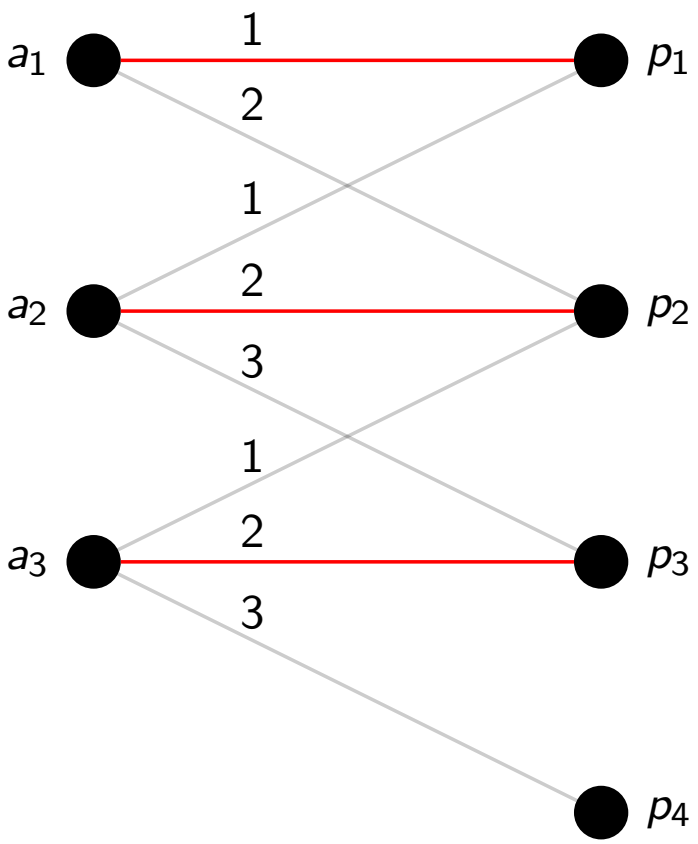
- A bipartite graph $G = (A \cup P, \mathcal{E})$
  - A denotes the set of applicants.
  - P denotes the set of posts.

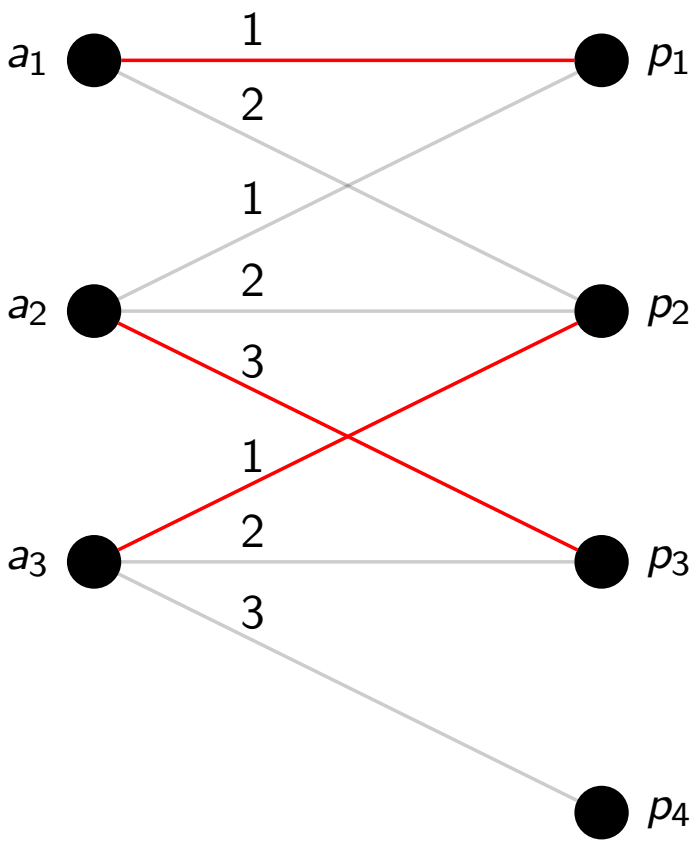


Goal:

A matching that matches the maximum number of applicants to their first choice, subject to that condition, the maximum number of applicants to their second choice, and so on.

Rank-Maximal Matching





Rank-Maximal Matchings: History

- Introduced by Irving as Greedy Matching in 1992.
- He gave a combinatorial algorithm for certain kind of preference lists.
- Generalized by Irving, Kavitha, Mehlhorn, Michail and Paluch to arbitrary preference lists in 2006.

Necessarily Rank-Maximal Matching

Necessarily Rank-Maximal Matching: Notations

Definition: Partial Preference List

An agent ranks *only a subset* $S \subsetneq O$ of objects in a strict order

$$o_1 \succ o_2 \succ \cdots \succ o_{|S|},$$

while objects in $O \setminus S$ remain unranked (incomparable).

Necessarily Rank-Maximal Matching: Notations

Definition: Partial Preference List

An agent ranks *only a subset* $S \subsetneq O$ of objects in a strict order

$$o_1 \succ o_2 \succ \cdots \succ o_{|S|},$$

while objects in $O \setminus S$ remain unranked (incomparable).

Definition: Completion of a Preference List

A *completion* of a partial list is any strict total order on O that:

- keeps the given order on the objects in S.
- places every object in S before every object in $O \setminus S$.
- places the unranked objects of $O \setminus S$ arbitrarily.

Necessarily Rank-Maximal matching

Motivation

Consider an instance where both N and O is very large, say $|N| = |O| > 100$. It is impractical to ask the full preferences of every agent.

Agents	**Preferences**			
a_1	o_1	o_2	$\cdots$	$\cdots$
a_2	o_1	o_2	$\cdots$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
a_n	o_3	o_4	$\cdots$	$\cdots$

Necessarily Rank-Maximal matching

Motivation

Consider an instance where both N and O is very large, say $|N| = |O| > 100$. It is impractical to ask the full preferences of every agent.

Agents	**Preferences**			
a_1	o_1	o_2	$\cdots$	$\cdots$
a_2	o_1	o_2	$\cdots$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
a_n	o_3	o_4	$\cdots$	$\cdots$

Goal

Compute a rank-maximal matching from **partial preference lists** of the agents that remains a rank-maximal matching under **all possible completions** of the partial preference lists.

Online Elicitation of NRM

Agents	Preferences		
a_1			
a_2			
a_3			

Agents	Preferences		
a_1	o_1		
a_2	o_1		
a_3	o_2		

Agents	Preferences		
a_1	o_1	o_3	
a_2	o_1	o_2	
a_3	o_2	o_3	

- Start with an empty preference profile
- Query agents about their preferences
- Stop eliciting once we obtained sufficient info
- Compute an rank-maximal matching
- **Competitive Ratio (α)**

$$\alpha = \frac{\#\text{ queries asked by the online algorithm}}{\#\text{ queries asked by the optimal offline algorithm}}$$

Query Models

- **Next-Best**: An agent on being queried returns the object they rank the highest among all objects that haven't been revealed for that agent yet. Denoted as $Q(a)$
- **Hybrid**: There are two kinds of queries
  1. $Q(a_i, k)$: The agent a_i returns the object at the kth position.
  2. $Q(a_i, o_j)$: The agent a_i returns the rank of object o_j.
- **Set-Compare**: Queries are of the form $Q(a_i, O')$ where $O' \subseteq O$. The agent a_i returns its most preferred object in O'

Results

Upper and Lower bounds for the best algorithms for NRM Elicitation

	Next-Best		Set-Compare		Hybrid	
	LB	UB	LB	UB	LB	UB
NRM	$\frac{3}{2}$	$\frac{3}{2}$	$\frac{3}{2}$	$O(n)$	$\frac{3}{2}$	6

Table: Comparison table with LB and UB under each query model

Open problem

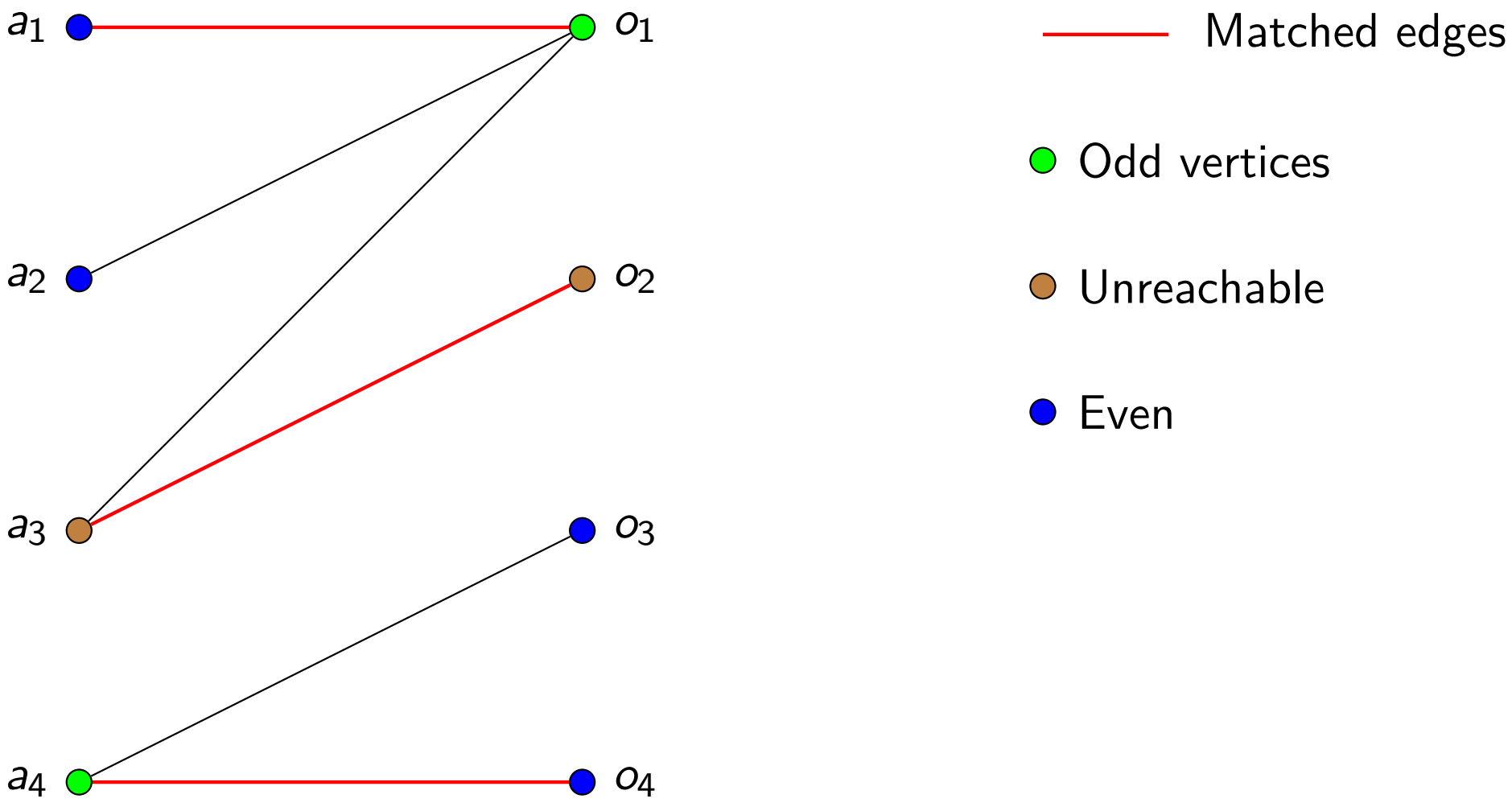
Upper Bound for Set Compare Model (Peters, 2022)

Find an online algorithm with a **constant or sublinear competitive ratio** for eliciting a *necessarily rank-maximal (NRM) matching* under the *Set-Compare query model*.

Gallai-Edmonds Decomposition

- Decomposes the set of vertices into three disjoint sets
  - $\mathcal{E}$ (Even) – set of vertices which can be reached from an unmatched (free) vertex via an alternating path of even length
  - $\mathcal{O}$ (Odd) – set of vertices which can be reached from an unmatched (free) vertex via an alternating path of odd length
  - $\mathcal{U}$ (Unreachable) – set of vertices which are unreachable from a free vertex via an alternating path

Gallai-Edmonds Decomposition



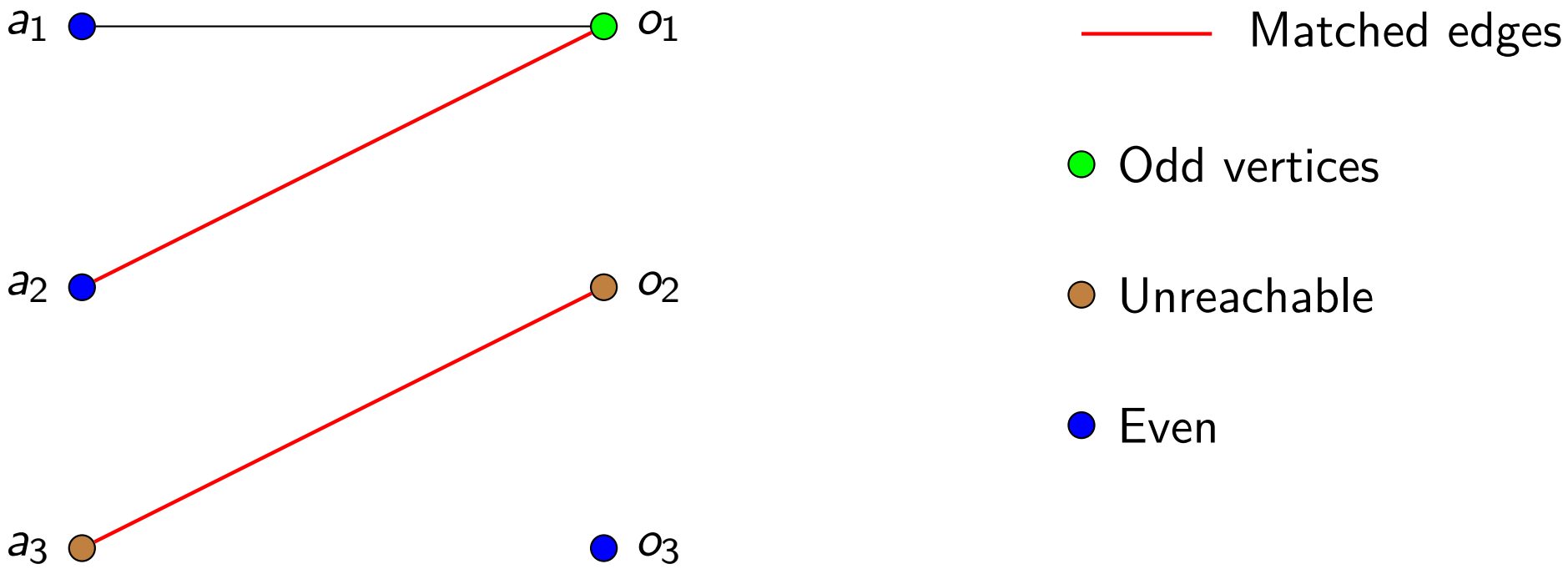
Modified Algorithm

- Elicit preferences in a 3-agent 3-object system

Agents	**1st**	**2nd**	**3rd**
a_1			
a_2			
a_3			

- Ask each agent their best object from $S = \{o_1, o_2, o_3\}$

Find a maximum matching in G



Modified Algorithm

- Elicit preferences in a 3-agent 3-object system

Agents	**1st**	**2nd**	**3rd**
a_1	o_1		
a_2	o_1		
a_3	o_2		

- $S_1 = \{o_3\}$
- $S_2 = \{o_3\}$
- $S_3 = \{o_3\}$
- Each agent prefers o_3 but the positions are unknown.

Modified Algorithm

- Elicit preferences in a 3-agent 3-object system

Agents	1st	2nd	3rd
a_1	o_1	o_3	
a_2	o_1	o_3	
a_3	o_2	o_3	

- **Can't o_3 be a_1's 3rd most preferred object as well?**

Modified Algorithm

- **This preference distribution gives the same query** outputs as before

Agents	1st	2nd	3rd
a_1	o_1		o_3
a_2	o_1	o_3	
a_3	o_2	o_3	

Modified Algorithm

Agents	1st	2nd	3rd
a_1	o_1	o_3	o_2
a_2	o_1	o_3	o_2
a_3	o_2	o_3	o_1

Agents	1st	2nd	3rd
a_1	o_1	o_2	o_3
a_2	o_1	o_3	o_2
a_3	o_2	o_3	o_1

- Both of the preference distributions give the same query outputs
- An NRM in one of them might not be an NRM in the other
- Observe that the lack of knowledge of rank is the issue

Rank-Aware Set Compare Model

Rank Aware Set Compare Model

The model asks the agent to reveal not just the most preferred object in the set but also the rank of the selected object:

$$Q(a_i, O') = (r, o), \quad O' \subseteq O$$

$$o = \arg\max_{o' \in O'} \mathrm{rank}(a_i, o), \qquad k = \mathrm{rank}(a_i, o)$$

with respect to $\succ_i$, the complete preference list of agent a_i.

Working of the algorithm

- H is a set of sets and each H_i is an agent-specific set containing the active objects for a_i
  - Delete objects in $\mathcal{O}$ or $\mathcal{U}$ from H_i for all i
  - Delete revealed objects for each agent
- Queries each a_i with its H_i and retrieves its best choice and the rank

Working of the algorithm (contd.)

- Maintains a dictionary D
  - If a_i gives the rank of the most liked object greater than the current iteration number, we store the rank-object pair in D
- At each iteration, check D for each agent a_i.
  - if D has the object for the current iteration, use it and clear from D
  - If D has a higher rank object, don't query a_i
  - If no entry for a_i, query the agent, and add the agent-object pair as an edge if it is not in the forbidden set or keep it in D.

Key Observations

- If an agent returns a higher rank object in an iteration, the objects in between the last revealed rank and this higher rank are inactive for this agent
- The queries for these ranks are futile and should be avoided

$\Downarrow$

a_1	o_1	×	×	×	o_5

- Let a_1 retire till 5th iteration

Elicitation Algorithm for NRM with Rank-Aware Set Compare Model - Initialization and Base Case

$A \leftarrow N$
$H \leftarrow [O] * n$
$E \leftarrow \phi$
$M \leftarrow \phi$
$F \leftarrow \phi$
$D \leftarrow \{\}$
if $|N| = 2$ **then**
  $h_1 = Q(a_1, H_1)$
  return $[a_1,\ h_1],\ [a_2,\ h_2]$
end if

Elicitation Algorithm for NRM with Rank-Aware Set Compare Model - Main loop

for i in range from 1 to $n-1$ **do**
  for a_i in A **do**
    $E = QueryAgent(a_i, D, H, i)$
  end for
  Augment M so that it is a maximum matching in $(N \cup O, E)$
  Calculate the Edmond-Gallai Decomposition $\mathcal{U}, \mathcal{E}, \mathcal{O}$ for M
  If agent $a \in N$ is $\mathcal{U}$ or $\mathcal{O}$, remove a from A
  If object $o \in O$ is $\mathcal{U}$ or $\mathcal{O}$, remove o from $H_i \forall i \in [n]$
  Add any $\mathcal{OO}$ or $\mathcal{OU}$ edges to F and remove them from E
end for
return M

$QueryAgent(a_i, D, H, i)$

```
skip ← False
h ← None
r ← None
if a_i in keys(D) then
    retrievedRank, retrievedObj ← D[a_i]
    if retrievedRank > i then
        skip ← True
    else if i = retrievedRank then
        h ← retrievedObj
        r ← retrievedRank
        skip ← True
        Remove a_i from D
    end if
end if
if skip = False then
    h, r ← Q(a_i, H_i)
    Remove h from H_i
end if
E = AddEdge(h, r, a_i, i, D)
return E
```

$AddEdge(h, r, a_i, i, D)$

```
if h is not None and r is not None then
    if (a_i, h) ∉ F and r = i then
        E = E ∪ {(a_i, h)}
    else if r > i then
        D[a_i] = (r, h)
    end if
end if
return E
```

Set-Compare Model: Example

Agents	Preferences				
a_1					
a_2					
a_3					
a_4					
a_5					

Query Sequence

$Q(a_1, O_1) = o_1$
$Q(a_2, O_2) = o_2$
$Q(a_3, O_3) = o_1$
$Q(a_4, O_4) = o_2$
$Q(a_5, O_5) = o_1$

Active Objects

O_1: o_1, o_2, o_3, o_4, o_5
O_2: o_1, o_2, o_3, o_4, o_5
O_3: o_1, o_2, o_3, o_4, o_5
O_4: o_1, o_2, o_3, o_4, o_5
O_5: o_1, o_2, o_3, o_4, o_5

Exemplary run

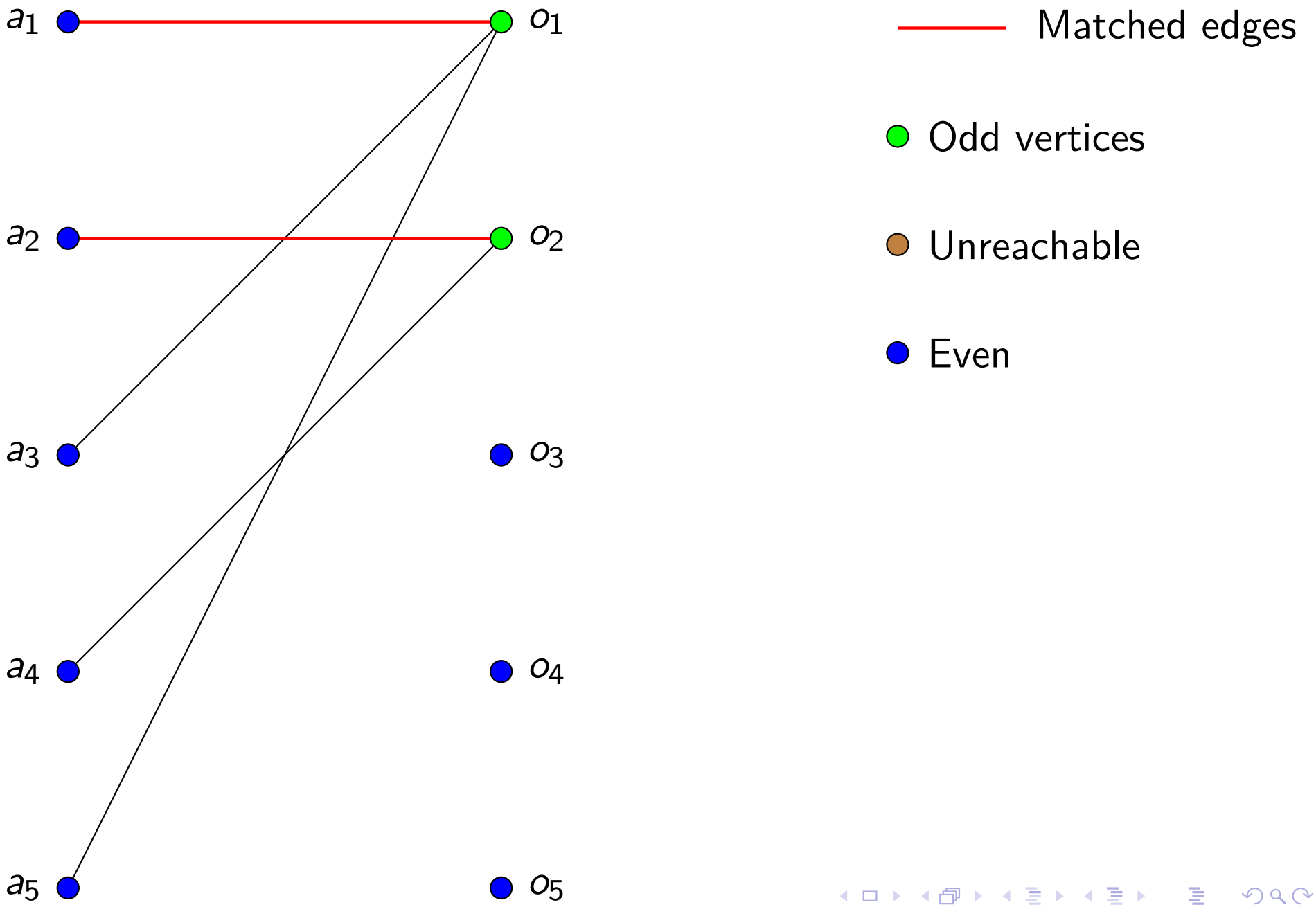
1st query round completed for each agent

Agents	**Preferences**				
a_1	o_1				
a_2	o_2				
a_3	o_1				
a_4	o_2				
a_5	o_1				

After each query round,

- Query model passes the information to the algorithm
- Find a maximum matching in $G = (N, O)$ from the revealed edges
- Decompose the vertices into $\mathcal{E}, \mathcal{O}, \mathcal{U}$
- Update the set of active agents and objects

Find a maximum matching in G.



Exemplary run

Dictionary

$D = \{\}$

Active Objects

O_1: o_3, o_4, o_5
O_2: o_3, o_4, o_5
O_3: o_3, o_4, o_5
O_4: o_3, o_4, o_5
O_5: o_3, o_4, o_5

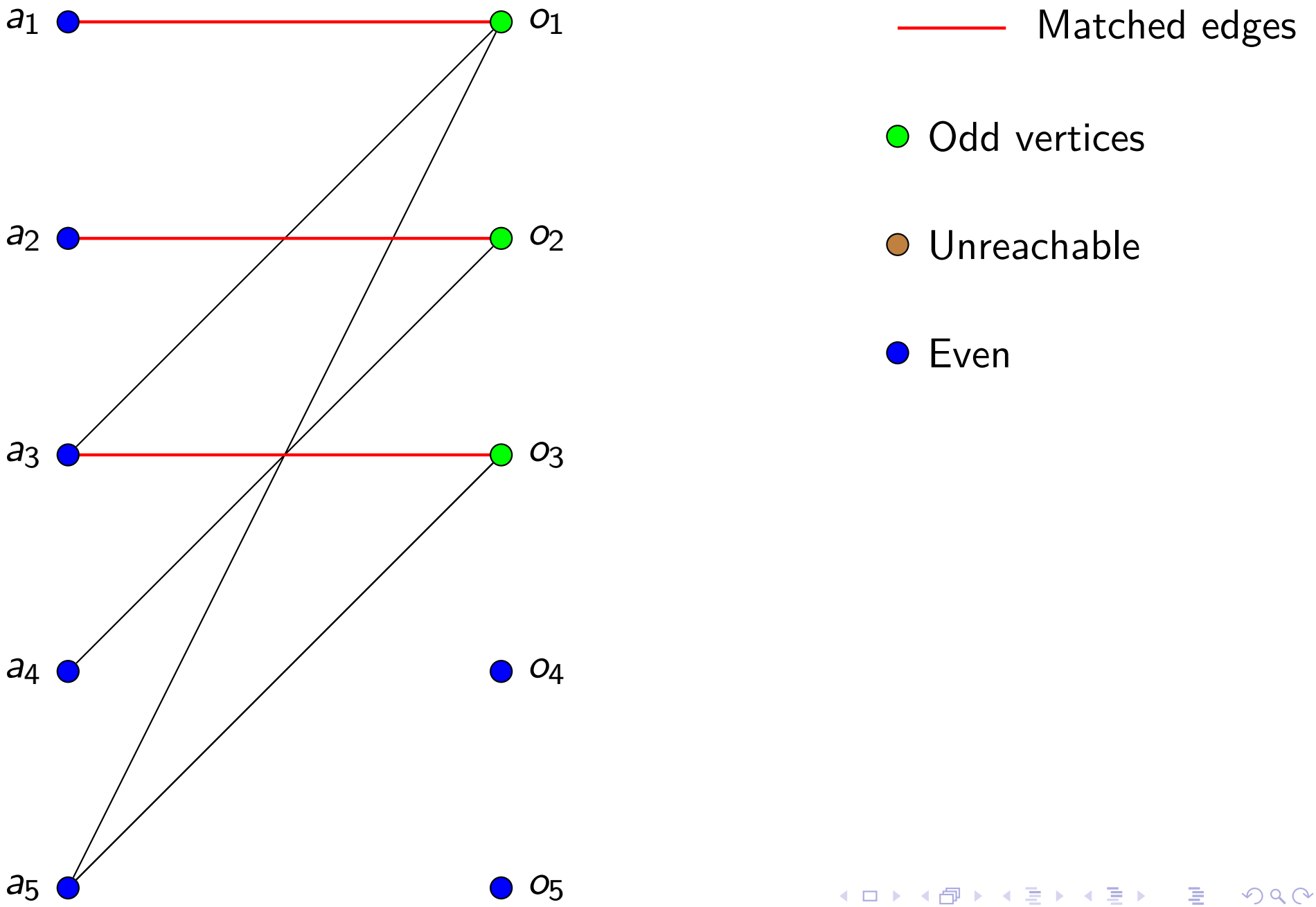
After second query round

Agents	Preferences				
a_1	o_1	X	o_3		
a_2	o_2	X	o_3		
a_3	o_1	o_3			
a_4	o_2	X	o_3		
a_5	o_1	o_3			

Agents	Preferences				
a_1	o_1	X	o_3		
a_2	o_2	X	o_3		
a_3	o_1	o_3			
a_4	o_2	X	o_3		
a_5	o_1	o_3			

The objects at the positions marked with a green cross need not be queried, because the best available rank for that agent is already greater than the deleted position, which corresponds to an inactive object.

Find a maximum matching in G.



Exemplary run

Dictionary

$D = \{a_1 : (o_3, 3), a_2 : (o_3, 3), a_4 : (o_3, 3)\}$

Active Objects

O_1: o_4, o_5
O_2: o_4, o_5
O_3: o_4, o_5
O_4: o_4, o_5
O_5: o_4, o_5

Before third iteration

Agents	Preferences				
a_1	o_1	X	o_3		
a_2	o_2	X	o_3		
a_3	o_1	o_3			
a_4	o_2	X	o_3		
a_5	o_1	o_3			

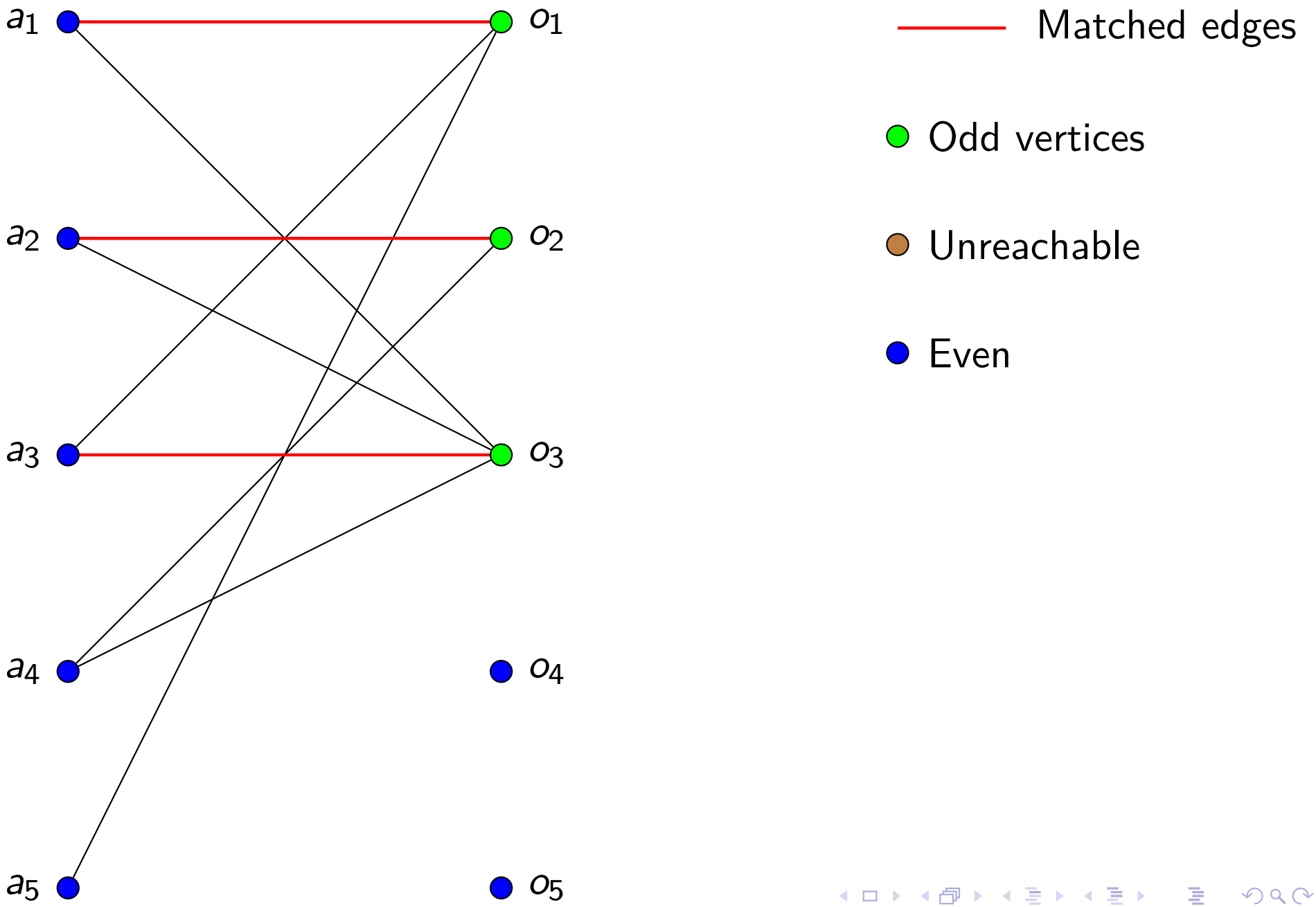
Exemplary run

After third iteration

Agents	Preferences				
a_1	o_1	X	o_3		
a_2	o_2	X	o_3		
a_3	o_1	o_3	X	o_5	
a_4	o_2	X	o_3		
a_5	o_1	o_3	X	o_4	

Those choices shaded in green are already revealed in second iteration and stored in memory.
Use this information
No queries are asked to these agents in this iteration.

Find a maximum matching in G



Exemplary run

Dictionary

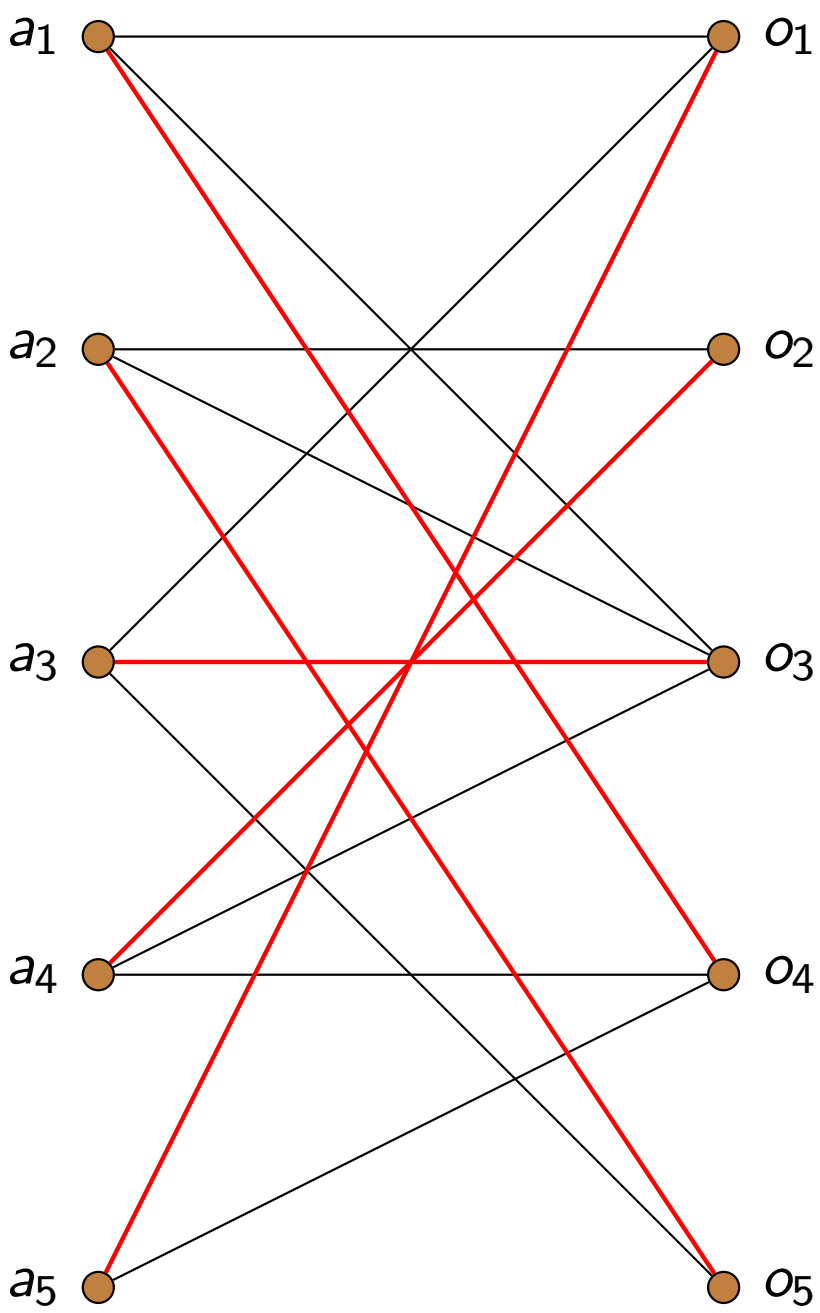
$D = \{a_3 : (o_5, 4), a_5 : (o_4, 4)\}$

Active Objects

O_1: o_4, o_5
O_2: o_4, o_5
O_3: o_4
O_4: o_4, o_5
O_5: o_5

Fourth iteration

Agents	Preferences				
a_1	o_1	X	o_3	o_4	
a_2	o_2	X	o_3	o_5	
a_3	o_1	o_3	X	o_5	
a_4	o_2	X	o_3	o_4	
a_5	o_1	o_3	X	o_4	



Matched edges

Odd vertices

Unreachable

Even

Dictionary

$D = \{\}$

Active Objects

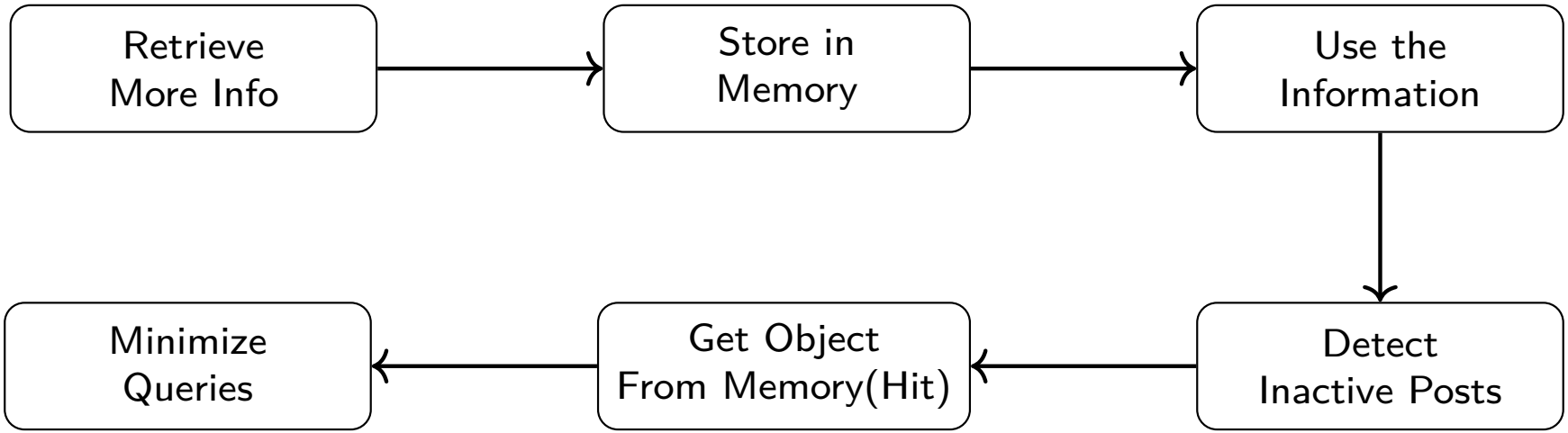
O_1: $\{\}$
O_2: $\{\}$
O_3: $\{\}$
O_4: $\{\}$
O_5: $\{\}$

- The above matching is an NRM
- We get an NRM with minimum number of queries

Agents	**Preferences**				
a_1	o_1	X	o_3	o_4	
a_2	o_2	X	o_3	o_5	
a_3	o_1	o_3	X	o_5	
a_4	o_2	X	o_3	o_4	
a_5	o_1	o_3	X	o_4	

- used 5 queries less than NBQM

Selling points in the algorithm



- The RASC query model retrieves more information than the vanilla model in each query.
- Minimizes the number of queries
- Additional memory of the next choice and its rank enables the algorithm to avoid unnecessary queries intelligently
- This cost of retrieval is covered up in the performance

Conjecture and Future Work

Conjecture

We conjecture that our modified query model admits an online algorithm with a **constant competitive ratio** for eliciting a necessarily rank-maximal matching.

Conjecture and Future Work

Conjecture

We conjecture that our modified query model admits an online algorithm with a **constant competitive ratio** for eliciting a necessarily rank-maximal matching.

Future Work

- Prove (or disprove) our conjecture.
- Design new query models for elicitation.
- Apply these query models to other optimality notions, such as *popular matchings*.

Thank you!